Reranking & recommendation-list ideas

Working notes on `src/rerank.py` and the recommendation slice in `starter/agent.py`. Ordered roughly by effort-to-payoff.

How reranking works today

`rerank()` (`src/rerank.py`) rescoring the top 200 of the retrieval pool:

```
        score(candidate) =  1.00 x span_coverage
                         +  1.00 x (bm25 / max_bm25_in_pool)      # normalised retrieval, 0..1
                         +  0.02 x popularity                      # tiebreak only

        span_coverage = sum over each disclosed constraint span:
                          (1 + 0.12 x word_count)   if the span is a literal substring of candidate["text"]
                          0                          otherwise
```

Then `sort(key=(-score, parent_asin))`, and pool ranks 201-300 are appended untouched.

Two properties that matter:

- **Per-item, no dependency.** Each candidate's score depends only on its own

text, its own BM25, its own rating count. There is no diversity penalty, no "demote because similar to a higher item", no pairwise learning-to-rank. The only shared quantity is `max_bm25_in_pool`, a uniform scaler.

- **Fully deterministic.** No randomness anywhere. Turn-to-turn changes in the

ranking come from `state.query_spans()` changing as new turns are observed (including junk spans from "I don't have an additional preference for X" replies), not from non-determinism.

Consequence: dropping ranks 1-10 and calling `rerank()` again on the rest returns essentially the same order - each score is independent of which other items are in the list. Re-running rerank buys nothing on its own.

Idea 1 - windowed recommendations (measured; superseded by 1b on this branch)

Instead of showing the same top 10 every turn 3-10, walk a frozen ranking in windows: turn 3 -> ranks 1-10, turn 4 -> 11-20, turn 5 -> 21-30, ... (`AgentConfig.scan_windows` on branch `kwongweng_rerank_once`; replaced by the elimination scan below on `kwongweng_elimination_scan`.)

- The ranking is frozen, then sliced. Freezing avoids a gap bug: if the live

pool reorders between turns, a target can fall between a window that was already passed and one not yet reached, and never be shown.

- Re-snapshot (and restart the scan from rank 1) whenever the information state

changes: a new *real* constraint, or an intent override taking effect. "Real" excludes the simulator's "no preference" replies, which also parse into a (useless) span - counting those re-froze the pool every turn and cancelled the whole effect. The override case matters because the evaluator ignores hits before the override, and the pre-override ranking often already has the target at rank 1 - without the re-freeze the scan sails past it.

- Depth: ~8 windows = pool ranks 1-80. Targets past rank 80 still unreachable

(rerank depth 200, pool 300) - see Idea 5.

Measured (`python3 -m evaluator.local_evaluator`, and `tools/hard_cases.py --run`)

set	metric	baseline	windowed	delta
public (200)	score	0.8592	**0.8907**	+0.032
public	hit@10	0.940	**0.990**	+0.050
public	MRR	0.7911	0.7942	+0.003

set	metric	baseline	windowed	delta
public	MTTC	3.41	3.13	-0.28
adversarial (96)	score	0.684	**0.764**	+0.080
adversarial	hit@10	0.740	**0.854**	+0.114

Per-scenario (public): browsing hit 0.963 -> 1.000, buying 0.938 -> 0.988, intent_override 0.867 -> 0.967, boundary hit 1.000 held (its MRR slipped 0.846 -> 0.756 - the two late-converting boundary sessions now hit a couple of ranks lower; ~0 net score impact). 29/29 tests pass, regression floor clears.

Idea 1b - elimination scan (branch `kwongweng_elimination_scan`)

A product shown on an earlier turn and *not* hit on is a confirmed non-target (the session would have ended). So each turn: drop everything already shown, return the top of the re-ranked *survivors*. `rerank()` runs every turn, so this reflects new constraints automatically - no frozen pool, cursor or re-snapshot signature. `AgentConfig.elimination_scan` (default on), `first_recommend_turn` is the start-turn knob, `hold_until_stalled` holds every list until a turn adds no new real constraint.

One correctness exception kept: a list shown *before* an intent override confirms nothing (the evaluator ignores hits until the override lands), so `_shown` is cleared on override detection and the scan restarts on the post-override ranking.

Sweep (`tools/sweep.py`, FixedPolicy, span rerank)

config	dev score	holdout score	note
plain (same top 10)	0.8738	0.8374	pre-scan floor
elim start turn 1	0.8691	0.8543	early hits freeze poor MRR (0.63-0.70)
elim start turn 2	0.8711	0.8648	same
elim start turn 3	**0.8967**	**0.8869**	**ships** - hit@10 0.98/1.00, MRR 0.82/0.78
elim start 2 + hold_until_stalled	0.8802	0.8652	best override MRR but MTTC ~4.1 kills efficiency

elim start-turn-3 vs windowed (full sets)

set	metric	windowed (1a)	elimination (1b)
public (200)	score	0.8907	**0.8928**
public	MRR	0.7942	**0.8041**
public	boundary MRR	0.756	**0.883** (windowing's regression, fixed)
public	intent_override	0.967 / MRR 0.725	1.000 / MRR 0.736
adversarial (96)	score	**0.764**	0.725
adversarial	hit@10	**0.854**	0.802

Elimination wins the real metric (public + holdout) and is simpler, but loses ~2-3 sessions on the adversarial stress set: when the disclosed constraints are all generic, the "no preference" junk spans make the ranking noisy enough that a borderline target (~rank 12-20) is pushed away before the shrinking survivor set reaches it - the frozen ranking in 1a can't do that. Most adversarial misses are just genuinely-too-deep targets (survivor rank 100-250 or ejected from the pool); both variants miss those. 29/29 tests pass.

Idea 1c - stop the "no preference" replies polluting retrieval (branch `kwongweng_retrieval_recall`)

Retrieval queries `state.full_text()` = every utterance joined. From turn 4 the simulator's *"I don't have an additional preference for feature / use_case / style / material / colour / size"* leaks `feature, style, material, colour, size, category` into the BM25 OR-query and the span matcher - terms that match huge swathes of the catalog and dilute the target.

Fix: `Utterance.declined` flag, set in `observe()` when `NO_PREFERENCE_CUES` matches; `full_text()`, `focused_text()` and `query_spans()` skip declined utterances. Also `RerankConfig.depth` 200 -> 300 (rescore the whole pool, not a prefix).

Retrieval recall (target in the pool - the point of the change)

	public end-of-session	adversarial end-of-session
in pool (300), before	98%	81%
in pool (300), after	**100%**	**95%**
in reranked top-200, before	97%	78%
after (depth 300)	**100%**	**95%**
end-of-session pool rank p90, before -> after	87 -> 16	279 -> 200

Per adversarial bucket, pool@end: degenerate_card 56 -> 88%, homogeneous_cluster 81 -> 100%, generic_override 75 -> 94%, cross_category 94 -> 100%. The within-session degradation is gone.

Score

set	before	after	delta
public (200)	0.8982	0.8995	+0.001 (hit 0.990 -> 0.995, MRR 0.821 -> 0.814)
dev	0.9003	0.9041	+0.004
holdout	0.8951	0.8925	**-0.003**
adversarial (96)	0.725	**0.794**	**+0.069** (hit 0.802 -> 0.885)

Recall goal achieved and a large adversarial gain, but on its own roughly flat on the public metric - the leaked decline terms were a weak tie-break that happened to favour a few already-well-ranked public targets (holdout MRR 0.80 -> 0.79).

Then merged with the teammate's facet-agreement rerank signal (Idea 2, `RerankConfig.facet_weight = 0.3`, `src/rerank.py`) - which recovers that MRR and more. Combined:

set	main	recall only	recall + facet
public	0.8982	0.8995	**0.9031**
public MRR	0.821	0.814	**0.825**
dev	0.9003	0.9041	**0.9087**
holdout	0.8951	0.8925	**0.8946** (flat)
adversarial	0.725	0.794	**0.788**
adversarial hit@10	0.802	0.885	**0.885**

Now a clear win: public + dev up, holdout flat, adversarial +0.06. The depth change is still a no-op on public; it only helps the adversarial set. 39/39 tests pass.

Idea 2 - richer rerank scoring

The reranker ignores signals that are already computed:

- **Category exact-match.** The opening message always names a coarse category.

Add `+ w_cat * (candidate_category_leaf == opening_category_leaf)`. `src/facets.py:_category_leaf()` already computes the leaf. Biggest expected gain - kills cross-category matches (a leather bag with a buckle competing with a leather belt).

- **Facet agreement + negative evidence.** `FacetStore` extracts

material/colour/size/brand/price/category for every product and no ranking signal reads any of it. Add `+ w_facet * agreement(candidate_facets, disclosed_facets)`, and a negative term: a candidate whose material is explicitly different from a stated one is pushed down, not merely left unrewarded.

- **Profile-tag affinity.** `user_profile.preference_tags` (["fit", "comfort",

"durability"]) is read only by the benched `InfoGainPolicy`. A small rerank term boosting candidates whose text resonates with the tags is the "safe personalization" the brief asks for. Weak on its own; useful stacked with the above.

These do not need windowing - they make the single ranking better, which helps every session.

Idea 3 - MMR diversity term (makes windowing productive)

Add a dependency term so re-ranking after a window is meaningful:

```
score(candidate) = relevance(candidate) - lambda * max_similarity(candidate, already_shown)
```

After showing ranks 1-10, re-scoring the rest with the shown items as "covered" pushes up candidates that are *different* - different brand, sub-style, price band. Each new window becomes a fresh slice of the plausible space rather than "the next 10 by the same score". For an ambiguous target (dozens of near-identical leather belts) this raises the hit chance materially. `IMPLEMENTATION.pdf` S7 - "Diversify a low-confidence list" - flags this as untested; the marginal-value table there (miss->hit ~= 2.6x a rank improvement) says trading rank for coverage on low-confidence sessions is likely net-positive.

Idea 4 - learn the weights

After Ideas 2-3, `rerank()` has ~6 hand-set weights (`span_weight`, `retrieval_weight`, `popularity_weight`, `length_bonus`, plus the new `w_cat` / `w_facet`). One offline logistic-regression pass over the ~176 known-correct public sessions fits them jointly - standard library, no network. `IMPLEMENTATION.pdf` S10 flags this as the highest-EV cross-cutting change; it should come *after* the new features exist.

Idea 5 - reach deeper than rank 80

If windowing helps but the depth ceiling bites:

- Widen later windows (turn 6+: show 21-40, 41-70, ...) instead of a flat 10.
- Raise `RerankConfig.depth` from 200 to 300 (= pool size) so no fused candidate

is left unscored in the tail. One-line change.

- Raise `RetrievalConfig.pool_size` for the first two turns.